The journey of finding a good way to do blogging

Ronaldo Gligan

2024-02-28

Índice

1. Appendix: *is blogging dead?* **1**

My old needs

- custom styling
- syntax highlighting
- easy deploy

My new needs

- those above
- good mathematical rendering

1. Chapter one: Wordpress
2. Chapter two: Jekyll
3. Chapter three: TeXmacs
4. Settling on Emacs

1. Appendix: *is blogging dead?*

I don't care… I'll still do it

$$\varphi f = f(\varphi f)$$

is the fix operator.

> 'Reading' means parsing an expression in textual form and producing a lisp object. E.g., this is a way to load a lisp file. 'Printing' a lisp object mean producing a textual representation. These operations, in lisp, are inverses.

Also, check out this OCaml snippet

```
open Printf
;;
(* type 'a ghostype *)

let double n = 2*n
;;
let () =
  let n = 145
  in printf "double %d = %d\n" n (double n)
```

```
double 145 = 290
```

Literate programming is a programming paradigm introduced in 1984 by Donald Knuth in which a computer program is given as an explanation of how it works in a natural language, such as English, interspersed (embedded) with snippets of macros and traditional source code, from which compilable source code can be generated.[1] The approach is used in scientific computing and in data science routinely for reproducible research and open access purposes.[2] Literate programming tools are used by millions of programmers today.[3]

The literate programming paradigm, as conceived by Donald Knuth, represents a move away from writing computer programs in the manner and order imposed by the compiler, and instead gives programmers macros to develop programs in the order demanded by the logic and flow of their thoughts.[4] Literate programs are written as an exposition of logic in more natural language in which macros are used to hide abstractions and traditional source code, more like the text of an essay.

Literate programming (LP) tools are used to obtain two representations from a source file: one understandable by a compiler or interpreter, the "tangled" code, and another for viewing as formatted documentation, which is said to be "woven" from the literate source.[5] While the first generation of literate programming tools were computer language-specific, the later ones are language-agnostic and exist beyond the individual programming languages.

```
-- Naïve solution that does not scale well
-- naive n = sum . map (\c -> fromEnum $ c == 'a') . take n . concat . repeat

-- Counts the number of 'a's given a string
countAs :: String -> Int
countAs = length . filter (== 'a')

-- Count the rest of a's
countRest :: Int -> String -> Int
countRest sub = countAs . take sub

solution :: Int -> String -> Int
solution n s =
  a * (n `div` l) + countRest r s
  where a = countAs s
        l = length s
        r = n `mod` l

main :: IO ()
```

```
main = do s <- getLine
          secondLine <- getLine
          let n = read secondLine
          putStrLn $ show $ solution n s
```

It is astonishing that one can do literate programming inside Org-mode.

```
print("hello")
# print(fileName, "is!")
```

```
None
```

Think of $\mathbb{K}_n[x]$ as the set of all degree n polynomials with an unknown x and coefficients on $\mathbb{K}$. Then, the application $I(p) = \int p \, \mathrm{d}x$ is a linear transformation and $I : \mathbb{K}_n[x] \longrightarrow \mathbb{K}_{n+1}[x]$. Furthermore, the derivative is also a linear application.